



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Andrews, Kurt Kent C.
Section:	AN41
Professor:	Ms. Crisola Tan
Date:	Oct 2, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF file**.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```

}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()

```

2. Run the program in your terminal.
3. Enter the following values when prompted:
 - **Number of agents** (e.g., 5)
 - **Grid size** (e.g., 10 for a 10x10 grid)
 - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:

- **More agents** (e.g., 10 or 20)
- **Smaller/larger grid sizes**
- **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths:** Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules:** Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions:** Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios:** Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ↓

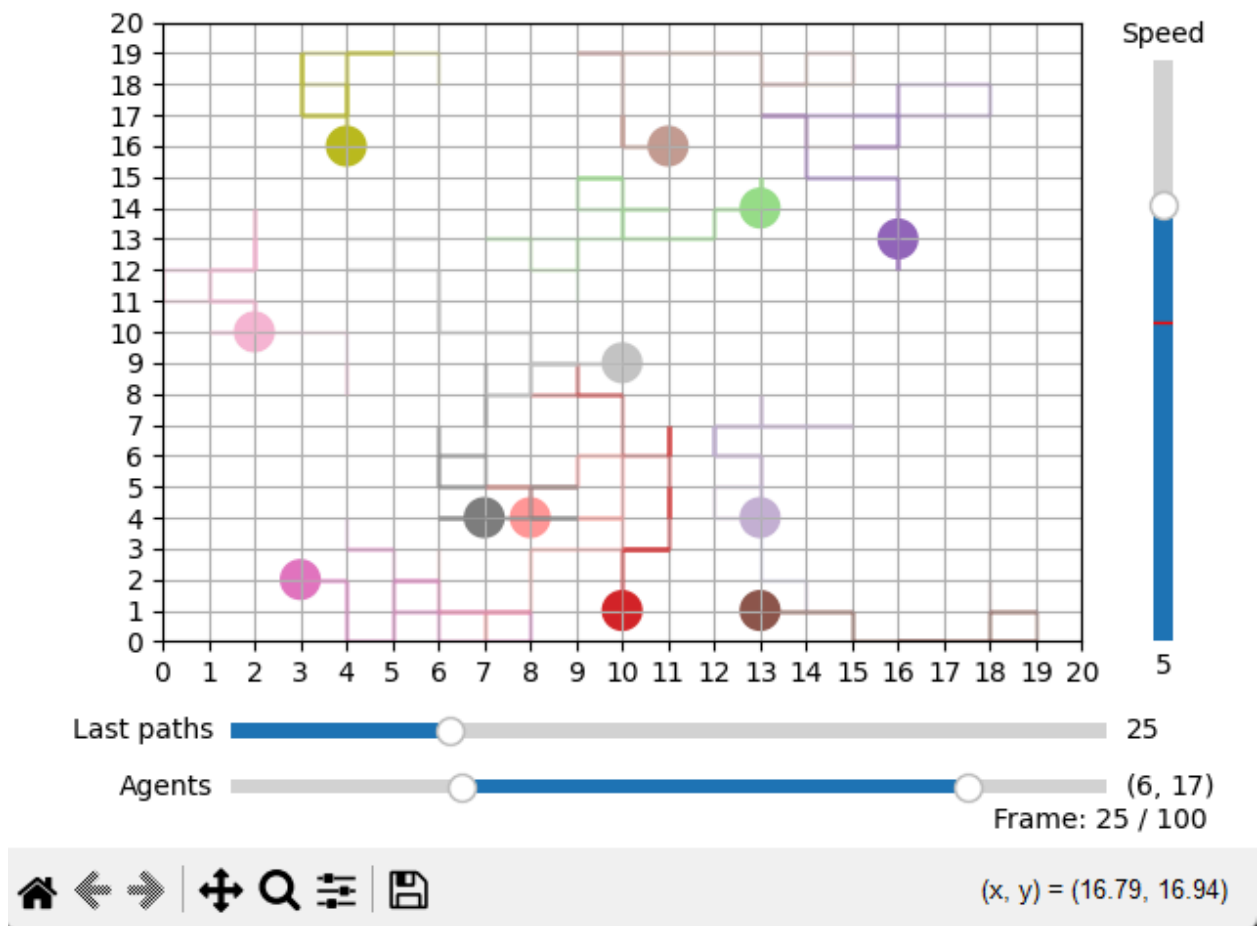
Code:

<https://github.com/UnlimitedLyfes/Andrews-KurtKent-CS0065-AN41>

Under TA1

```
Enter number of agents: 20
Enter grid size (e.g., 10 for 10x10): 20
Enter number of steps: 100
Movement mode (random/avoid/preferred/nooverlap): nooverlap
```

Figure 1



```
Final position analysis
-----
Movement mode: nooverlap
Agents: 20
Occupied cells: 20 / 400
Agents sharing cells: 0
Densest cell: (9, 0) with 1 agent(s)
Mean final position: (9.25, 8.60)
Final position variance: x=28.41, y=24.25
X range: 1 to 17
Y range: 0 to 18
Saved final positions to final_positions.csv
```

Step 5: Answer the Questions

For context, in the implementation, there are four movement rules that can be chosen (originally three, but “Make agents avoid each other” seemed ambiguous, and the use of “or” implied the user is given the choice among the movement rules. Additionally, I have made initial positions unique, and, if agents exceed the edge, uniform.

- Random - Default, agents move without considering other agents as long as they are within the grid's boundaries.
- Avoid - Agents actively choose which positions move them farthest from each other. Overlap can still happen if the agent is left with no choice.
- Nooverlap - Agents move randomly, but will choose not to overlap with other agents. If left with no choice, will stay still to preserve the “no overlap” rule.
- Preferred - Agents have a bias to move towards a certain direction of 75% (preferred direction has weight of 6, others have weight of 1: 6/9 chance to move to preferred direction).

1. What happens if the number of agents is **very high** compared to the grid size?

It highly depends on the movement rules. With the default movement rules of allowed collisions and no biases, the simulation proceeds normally, except that the odds of the edge having an agent on it would be higher due to the high agent-to-edge ratio. For movement rules avoid and nooverlap, the agents would have very limited movement, likely going in circles, and once agents completely overwhelm the number of edges, there would be seemingly no movement. For preferred directions, there is not much of a difference, as the agents would still end up hugging the edges after a certain number of steps have passed.

2. How does increasing the number of steps affect the **spread of agents**?

Random - Seemingly no difference, likely because the average displacement of all the agents is zero. I have further researched and hypothesized about this phenomenon, and initial findings hint that there should be an increase in the “spread-outness” of the agents, but after further analysis, I have found that this is only true in the case that the agents start at low entropy. This means that it does not apply to the current simulation, since the agents are placed randomly across the entire grid, uniformly. The spread would only increase if the agents' initial “spawn” is contained within less than 100% of the bounding grid, which is the case for many simulations.

Avoid - As steps increase, the spread increases, hitting a certain ceiling once the agents are in their farthest position from each other (because the grid is bounded).

Preferred - As steps increase, the spread decreases, hitting the floor once the minimum spread is reached (when all agents “hug” the wall of the direction they are biased to).

No Overlap - Similar to random, no difference. This would likely only have a difference if the initial positions were not set to be unique.

3. Do the agents ever move outside the grid? Why or why not?

No, the agents are hardcoded to never make moves towards the outside of the grid, even when no other directions can be travelled to (“Avoid” agents make an overlap, and “Nooverlap” agents stay still).

4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

There are too many factors to account for for the current implementation to pose a real similarity to crowd movement. “Random” mode, for example, cannot simulate even people who were told to “move randomly” due to innate psychological biases, similar to the Blue-seven Phenomenon, where the number 7 is most picked as a “random” number. In all the implemented modes, “Avoid” mode likely has the best chance among the four to model real-life phenomena, as it is similar to the real-life phenomenon of people attempting to form the biggest personal bubble in a crowded area; however, it cannot be accounted for in a non-crowded area, as the attempt to form a big personal

bubble is often affected by other factors such as effort.

However, this does not mean that simulations cannot model real-life phenomena. Crowd movement is simply very complex and would require a more robust implementation and rules. Other fields, especially those that use non-living objects, can be more easily modeled using simulations similar to this one. For example, those that deal with magnetism- repulsion and attraction- can be modeled by taking into account polarities and magnitude.

Answer the Questions 